EXPERIMENT 01 — Implementation of Basic Array Operations (Insertion, Deletion, Traversal)

Source filename: 11259A477exp1.c

Aim: To write a C program to perform basic operations on a one-dimensional array such as insertion of an element at a given position, deletion of an element from a given position, and traversal of the array.

Corrected C source code

```
#include <stdio.h>

#define MAX 100

void traverse(int arr[], int n) {
    int i;
    printf("Array elements: ");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int insert(int arr[], int n, int pos, int value) {
    int i;
    if (n >= MAX) {
        printf("Array is full. Cannot insert.\n");

            return n;
        }
        if (pos < 0 || pos > n) {
            printf("Invalid position.\n");
            return n;
        }
        for (i = n; i > pos; i--)
            arr[i] = arr[i - 1];
        arr[pos] = value;
        return n + 1;
}

int deleteElement(int arr[], int n, int pos) {
    int i;
    if (pos < 0 || pos >= n) {
        printf("Invalid position.\n");
        return n;
    }
    for (i = pos; i < n - 1; i++)
        arr[i] = arr[i + 1];
    return n - 1;
}

int main() {
    int arr[MAX], n, choice, pos, value;

        printf("Enter number of elements: ");
        scanf("%d", &n);
        printf("Enter %d elements: ", n);
        for (int i = 0; i < n; i++)
            scanf("%d", &arr[i]);

        do {
               printf("\n--- Array Operations Menu ---\n");
               printf("1. Insert\n2. Delete\n3. Traverse\n4. Exit\n");
               printf("Enter your choice: ");
               scanf("%d", &choice);

               switch (choice) {
                   case 1:
                       printf("Enter position (0-indexed) and value to insert: ");
                       scanf("%d %d", &pos, &value);
                       n = insert(arr, n, pos, value);
                       traverse(arr, n);
                       break;

                case 2:
                    printf("Enter position (0-indexed) to delete: ");
                    scanf("%d", &pos);
                    n = deleteElement(arr, n, pos);
                    traverse(arr, n);
                    break;
                case 3:
                    traverse(arr, n);
                    break;
                case 4:
                    printf("Exiting program.\n");
                    break;
```

```
                default:
                    printf("Invalid choice.\n");
            }
        } while (choice != 4);

        return 0;
}
```

EXPERIMENT 01 — RUN OUTPUT

Output screenshot name: 11259A477exp1_output.png

OUTPUT | 11259A477 exp1.c

```
Test case: n = 6 | Initial array: 10 20 30 40 50 60 | Insert (position 3, value 99) | Delete position 1

Enter number of elements: Enter 6 elements:
--- Array Operations Menu ---
1. Insert
2. Delete
3. Traverse
4. Exit
Enter your choice: Enter position (0-indexed) and value to insert: Array elements: 10 20 30 99 40 50 60

--- Array Operations Menu ---
1. Insert
2. Delete
3. Traverse
4. Exit
Enter your choice: Enter position (0-indexed) to delete: Array elements: 10 30 99 40 50 60

--- Array Operations Menu ---
1. Insert
2. Delete
3. Traverse
4. Exit
Enter your choice: Array elements: 10 30 99 40 50 60

--- Array Operations Menu ---
1. Insert
2. Delete
3. Traverse
4. Exit
Enter your choice: Exiting program.
```

Verification: Program compiled successfully and the output shown was captured from its execution using the test case printed above the output area.

EXPERIMENT 02 — Building a Dynamic Employee Record System using Structure Pointers

Source filename: 11259A477exp2.c

Aim: To write a C program to create a dynamic employee record system using structures and pointers, which allows storing, displaying, and searching employee records at run time using dynamic memory allocation.

Corrected C source code

```
#include <stdio.h>
 #include <stdlib.h>
 #include <string.h>

 struct Employee {
     int id;
     char name[50];
     float salary;
 };

 int main() {
     int n, i, choice;
     struct Employee *emp;

         printf("Enter number of employees: ");
         scanf("%d", &n);

         emp = (struct Employee *) malloc(n * sizeof(struct Employee));
         if (emp == NULL) {
             printf("Memory allocation failed.\n");
             return 1;
         }

         for (i = 0; i < n; i++) {
             printf("\nEnter details for employee %d\n", i + 1);
             printf("ID: ");
             scanf("%d", &(emp + i)->id);
             printf("Name: ");
             scanf("%s", (emp + i)->name);
             printf("Salary: ");
             scanf("%f", &(emp + i)->salary);
         }

         do {
         printf("\n--- Employee Record Menu ---\n");
         printf("1. Display all employees\n2. Search employee by ID\n3. Exit\n");
         printf("Enter your choice: ");
         scanf("%d", &choice);

         if (choice == 1) {
             printf("\n%-6s %-20s %-10s\n", "ID", "Name", "Salary");
             for (i = 0; i < n; i++)
                 printf("%-6d %-20s %-10.2f\n", (emp + i)->id, (emp + i)->name, (emp + i)->salary);
         } else if (choice == 2) {
             int searchId, found = 0;
             printf("Enter ID to search: ");
             scanf("%d", &searchId);
             for (i = 0; i < n; i++) {
                 if ((emp + i)->id == searchId) {
                     printf("Found: ID=%d Name=%s Salary=%.2f\n",
                            (emp + i)->id, (emp + i)->name, (emp + i)->salary);
                     found = 1;
                     break;
                 }
             }
             if (!found)

                     printf("Employee with ID %d not found.\n", searchId);
             }
         } while (choice != 3);

         free(emp);
         return 0;
 }
```

EXPERIMENT 02 — RUN OUTPUT

Output screenshot name: 11259A477exp2_output.png

OUTPUT | 11259A477 exp2.c

Test case: n = 3 employees | IDs 101, 102, 103 | Display all | Search ID 102

```
Enter number of employees:
Enter details for employee 1
ID: Name: Salary:
Enter details for employee 2
ID: Name: Salary:
Enter details for employee 3
ID: Name: Salary:
--- Employee Record Menu ---
1. Display all employees
2. Search employee by ID
3. Exit
Enter your choice:
ID     Name                 Salary
101    Arun                 45000.00
102    Bala                 52000.00
103    Chitra               48000.00

--- Employee Record Menu ---
1. Display all employees
2. Search employee by ID
3. Exit
Enter your choice: Enter ID to search: Found: ID=102 Name=Bala Salary=52000.00

--- Employee Record Menu ---
1. Display all employees
2. Search employee by ID
3. Exit
Enter your choice:
```

Verification: Program compiled successfully and the output shown was captured from its execution using the test case printed above the output area.

EXPERIMENT 03 — Implementation of Stack and Expression Conversion - Infix to Postfix

Source filename: 11259A477exp3.c

Aim: To write a C program to implement a stack using arrays (PUSH, POP, PEEK operations) and to convert a given infix expression into its equivalent postfix expression using a stack.

Corrected C source code

```
#include <stdio.h>
 #include <string.h>
 #include <ctype.h>

 #define MAX 100

 char stack[MAX];
 int top = -1;

 void push(char c) {
     if (top == MAX - 1)
          printf("Stack Overflow\n");
     else
          stack[++top] = c;
 }

 char pop() {
     if (top == -1) {
         printf("Stack Underflow\n");
         return -1;
     }
     return stack[top--];
 }

 char peek() {
     if (top == -1)
         return -1;
     return stack[top];
 }

 int precedence(char c) {
     if (c == '^') return 3;
     if (c == '*' || c == '/') return 2;
     if (c == '+' || c == '-') return 1;
     return -1;
 }

 int isOperand(char c) {
     return isalnum(c);
 }

 void infixToPostfix(char infix[], char postfix[]) {
     int i, j = 0;
     for (i = 0; infix[i] != '\0'; i++) {
         char c = infix[i];
         if (isOperand(c)) {
             postfix[j++] = c;
         } else if (c == '(') {
             push(c);
         } else if (c == ')') {
             while (top != -1 && peek() != '('

                      postfix[j++] = pop();
                 pop(); /* remove '(' */
             } else {
                 while (top != -1 && precedence(peek()) >= precedence(c))
                      postfix[j++] = pop();
                 push(c);
             }
         }
         while (top != -1)
             postfix[j++] = pop();
         postfix[j] = '\0';
 }

 int main() {
     char infix[MAX], postfix[MAX];
     printf("Enter an infix expression: ");
     scanf("%s", infix);
     infixToPostfix(infix, postfix);
     printf("Postfix expression: %s\n", postfix);
     return 0;
 }
```

EXPERIMENT 03 — RUN OUTPUT

Output screenshot name: 11259A477exp3_output.png

```
OUTPUT  |  11259A477 exp3.c
```

```
Test case: Infix input: A+B*(C-D)/E

Enter an infix expression: Postfix expression: ABCD-*E/+
```

Verification: Program compiled successfully and the output shown was captured from its execution using the test case printed above the output area.

EXPERIMENT 04 — Postfix Expression Evaluation using Stack

Source filename: 11259A477exp4.c

Aim: To write a C program to evaluate a given postfix expression using a stack.

Corrected C source code

```
#include <stdio.h>
 #include <string.h>
 #include <ctype.h>
 #include <math.h>

 #define MAX 100

 float stack[MAX];
 int top = -1;

 void push(float val) {
     if (top == MAX - 1)
          printf("Stack Overflow\n");
     else
          stack[++top] = val;

 }

 float pop() {
     if (top == -1) {
         printf("Stack Underflow\n");
         return 0;
     }
     return stack[top--];
 }

 float evaluatePostfix(char postfix[]) {
     int i;
     float op1, op2, result;

         for (i = 0; postfix[i] != '\0'; i++) {
             char c = postfix[i];

             if (isdigit(c)) {
                 push(c - '0');
             } else if (isspace(c)) {
                 continue;
             } else {
                 op2 = pop();
                 op1 = pop();
                 switch (c) {
                      case '+': result = op1 + op2; break;
                      case '-': result = op1 - op2; break;
                      case '*': result = op1 * op2; break;
                      case '/': result = op1 / op2; break;
                      case '^': result = pow(op1, op2); break;
                      default:
                          printf("Invalid operator: %c\n", c);
                          result = 0;
                 }
                 push(result);
             }
         }
         return pop();
 }

 int main() {
     char postfix[MAX];
     printf("Enter a postfix expression (single-digit operands): ");
     scanf("%s", postfix);
     printf("Result = %.2f\n", evaluatePostfix(postfix));

         return 0;
 }
```

EXPERIMENT 04 — RUN OUTPUT

Output screenshot name: 11259A477exp4_output.png

```
OUTPUT  |  11259A477 exp4.c
```

```
Test case: Postfix input: 623*-8+

Enter a postfix expression (single-digit operands): Result = 8.00
```

Verification: Program compiled successfully and the output shown was captured from its execution using the test case printed above the output area.

EXPERIMENT 05 — Implementation of Simple Queue using Array

Source filename: 11259A477exp5.c

Aim: To write a C program to implement a simple (linear) queue using an array and to perform ENQUEUE, DEQUEUE, and display operations.

Corrected C source code

```
#include <stdio.h>

#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

void enqueue(int val) {
    if (rear == MAX - 1) {
        printf("Queue Overflow\n");
        return;
    }
    if (front == -1)
        front = 0;
    queue[++rear] = val;
    printf("%d enqueued to queue.\n", val);
}

void dequeue() {
    if (front == -1 || front > rear) {
        printf("Queue Underflow\n");
        return;
    }
    printf("%d dequeued from queue.\n", queue[front]);
    front++;
    if (front > rear)
        front = rear = -1;
}

void display() {
    int i;
    if (front == -1 || front > rear) {
        printf("Queue is empty.\n");
        return;
    }
    printf("Queue elements: ");
    for (i = front; i <= rear; i++)
        printf("%d ", queue[i]);
    printf("\n");
}

int main() {
    int choice, val;
    do {
         printf("\n--- Simple Queue Menu ---\n");
         printf("1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");
         printf("Enter your choice: ");
         scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to enqueue: ");
                scanf("%d", &val);
                enqueue(val);
                break;
            case 2:
                dequeue();
                break;
            case 3:
                display();
                break;
            case 4:
                printf("Exiting program.\n");

                    break;
                default:
                    printf("Invalid choice.\n");
            }
        } while (choice != 4);
        return 0;
}
```

EXPERIMENT 05 — RUN OUTPUT

Output screenshot name: 11259A477exp5_output.png

OUTPUT | 11259A477 exp5.c

```
Test case: Enqueue 10, 20, 30 | Display | Dequeue | Display

--- Simple Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: Enter value to enqueue: 10 enqueued to queue.

--- Simple Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: Enter value to enqueue: 20 enqueued to queue.

--- Simple Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: Enter value to enqueue: 30 enqueued to queue.

--- Simple Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: Queue elements: 10 20 30

--- Simple Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 10 dequeued from queue.

--- Simple Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: Queue elements: 20 30

--- Simple Queue Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: Exiting program.
```

Verification: Program compiled successfully and the output shown was captured from its execution using the test case printed above the output area.

EXPERIMENT 06 — Implementation of Priority Queue using Unordered Array

Source filename: 11259A477exp6.c

Aim: To write a C program to implement a Priority Queue using an unordered array and to perform FindMin, Insert, and DeleteMin operations, where the element with the minimum value has the highest priority.

Corrected C source code

```
#include <stdio.h>

#define MAX 100

int pq[MAX];
int n = 0;

void insert(int val) {
    if (n == MAX) {
        printf("Priority Queue is full.\n");
        return;
    }
    pq[n++] = val;
    printf("%d inserted into priority queue.\n", val);
}

int findMinIndex() {
    int i, minIndex = 0;

        for (i = 1; i < n; i++)
            if (pq[i] < pq[minIndex])
                minIndex = i;
        return minIndex;
}

void findMin() {
    if (n == 0) {
        printf("Priority Queue is empty.\n");
        return;
    }
    printf("Minimum element (highest priority): %d\n", pq[findMinIndex()]);
}

void deleteMin() {
    int i, minIndex;
    if (n == 0) {
         printf("Priority Queue is empty. Cannot delete.\n");
         return;
    }
    minIndex = findMinIndex();
    printf("Deleted min element: %d\n", pq[minIndex]);
    for (i = minIndex; i < n - 1; i++)
         pq[i] = pq[i + 1];
    n--;
}

void display() {
    int i;
    if (n == 0) {
        printf("Priority Queue is empty.\n");
        return;
    }
    printf("Priority Queue elements: ");
    for (i = 0; i < n; i++)
        printf("%d ", pq[i]);
    printf("\n");
}

int main() {
    int choice, val;
    do {
         printf("\n--- Priority Queue Menu ---\n");
         printf("1. Insert\n2. FindMin\n3. DeleteMin\n4. Display\n5. Exit\n");

           printf("Enter your choice: ");
           scanf("%d", &choice);

            switch (choice) {
                case 1:
                    printf("Enter value to insert: ");
                    scanf("%d", &val);
                    insert(val);
                    break;
                case 2:
                    findMin();
```

```
                break;
            case 3:
                deleteMin();
                break;
            case 4:
                display();
                break;
            case 5:
                printf("Exiting program.\n");
                break;
            default:
                printf("Invalid choice.\n");
        }
    } while (choice != 5);
    return 0;
}
```

EXPERIMENT 06 — RUN OUTPUT

Output screenshot name: 11259A477exp6_output.png

OUTPUT | 11259A477 exp6.c

Test case: Insert 40, 10, 25 | Display | FindMin | DeleteMin | Display

```
--- Priority Queue Menu ---
1. Insert
2. FindMin
3. DeleteMin
4. Display
5. Exit
Enter your choice: Enter value to insert: 40 inserted into priority queue.

--- Priority Queue Menu ---
1. Insert
2. FindMin
3. DeleteMin
4. Display
5. Exit
Enter your choice: Enter value to insert: 10 inserted into priority queue.

--- Priority Queue Menu ---
1. Insert
2. FindMin
3. DeleteMin
4. Display
5. Exit
Enter your choice: Enter value to insert: 25 inserted into priority queue.

--- Priority Queue Menu ---
1. Insert
2. FindMin
3. DeleteMin
4. Display
5. Exit
Enter your choice: Priority Queue elements: 40 10 25

--- Priority Queue Menu ---
1. Insert
2. FindMin
3. DeleteMin
4. Display
5. Exit
Enter your choice: Minimum element (highest priority): 10

--- Priority Queue Menu ---
1. Insert
2. FindMin
3. DeleteMin
4. Display
5. Exit
Enter your choice: Deleted min element: 10

--- Priority Queue Menu ---
1. Insert
2. FindMin
3. DeleteMin
4. Display
5. Exit
Enter your choice: Priority Queue elements: 40 25

--- Priority Queue Menu ---
1. Insert
2. FindMin
3. DeleteMin
4. Display
5. Exit
Enter your choice: Exiting program.
```

Verification: Program compiled successfully and the output shown was captured from its execution using the test case printed above the output area.

EXPERIMENT 07 — Implementation of Singly Linked List (Insertion, Deletion, and Traversal)

Source filename: 11259A477exp7.c

Aim: To write a C program to implement a singly linked list and perform insertion (at the beginning, at a given position, and at the end), deletion (at the beginning, at a given position, and at the end), and traversal of the list.

Corrected C source code

```
#include <stdio.h>
 #include <stdlib.h>

 struct Node {
     int data;

         struct Node *next;
 };

 struct Node *head = NULL;

 void insertBeginning(int val) {
     struct Node *newNode = (struct Node *) malloc(sizeof(struct Node));
     newNode->data = val;
     newNode->next = head;
     head = newNode;
 }

 void insertEnd(int val) {
     struct Node *newNode = (struct Node *) malloc(sizeof(struct Node));
     newNode->data = val;
     newNode->next = NULL;
     if (head == NULL) {
         head = newNode;
         return;
     }
     struct Node *temp = head;
     while (temp->next != NULL)
         temp = temp->next;
     temp->next = newNode;
 }

 void insertPosition(int val, int pos) {
     if (pos == 0) {
         insertBeginning(val);
         return;
     }
     struct Node *newNode = (struct Node *) malloc(sizeof(struct Node));
     newNode->data = val;
     struct Node *temp = head;
     int i;
     for (i = 0; i < pos - 1 && temp != NULL; i++)
         temp = temp->next;
     if (temp == NULL) {
         printf("Invalid position.\n");
         free(newNode);
         return;
     }
     newNode->next = temp->next;
     temp->next = newNode;
 }

 void deleteBeginning() {
     if (head == NULL) {
         printf("List is empty.\n");
         return;
     }
     struct Node *temp = head;
     head = head->next;
     free(temp);
 }

 void deleteEnd() {
     if (head == NULL) {
         printf("List is empty.\n");
         return;
     }
     if (head->next == NULL) {
         free(head);
         head = NULL;
         return;
     }
     struct Node *temp = head;
     while (temp->next->next != NULL)
         temp = temp->next;
```

```
    free(temp->next);
    temp->next = NULL;
}

void deletePosition(int pos) {
    if (pos == 0) {
        deleteBeginning();
        return;
    }
    struct Node *temp = head;
    int i;
    for (i = 0; i < pos - 1 && temp != NULL; i++)
        temp = temp->next;
    if (temp == NULL || temp->next == NULL) {
        printf("Invalid position.\n");
        return;
    }
    struct Node *toDelete = temp->next;
    temp->next = toDelete->next;
    free(toDelete);
}

void traverse() {
    struct Node *temp = head;
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    int choice, val, pos;
    do {
         printf("\n--- Singly Linked List Menu ---\n");
         printf("1. Insert at Beginning\n2. Insert at Position\n3. Insert at End\n");
         printf("4. Delete at Beginning\n5. Delete at Position\n6. Delete at End\n");
         printf("7. Traverse\n8. Exit\n");
         printf("Enter your choice: ");
         scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value: ");
                scanf("%d", &val);
                insertBeginning(val);
                break;
            case 2:
                printf("Enter value and position (0-indexed): ");
                scanf("%d %d", &val, &pos);
                insertPosition(val, pos);
                break;
            case 3:
                printf("Enter value: ");
                scanf("%d", &val);
                insertEnd(val);
                break;
            case 4:
                deleteBeginning();

                    break;
                case 5:
                    printf("Enter position (0-indexed): ");
                    scanf("%d", &pos);
                    deletePosition(pos);
                    break;
                case 6:
                    deleteEnd();
                    break;
                case 7:
                    traverse();
                    break;
                case 8:
                    printf("Exiting program.\n");
                    break;
                default:
                    printf("Invalid choice.\n");
            }
        } while (choice != 8);
        return 0;
}
```

EXPERIMENT 07 — RUN OUTPUT

Output screenshot name: 11259A477exp7_output.png

OUTPUT | 11259A499exp7.c

Test case: Insert 10, 20, 30 at end | Insert 15 at position 1 | Traverse | Delete position 1 | Traverse

```
--- Singly Linked List Menu ---
1. Insert at Beginning
2. Insert at Position
3. Insert at End
4. Delete at Beginning
5. Delete at Position
6. Delete at End
7. Traverse
8. Exit
Enter your choice: Enter value:
--- Singly Linked List Menu ---
1. Insert at Beginning
2. Insert at Position
3. Insert at End
4. Delete at Beginning
5. Delete at Position
6. Delete at End
7. Traverse
8. Exit
Enter your choice: Enter value:
--- Singly Linked List Menu ---
1. Insert at Beginning
2. Insert at Position
3. Insert at End
4. Delete at Beginning
5. Delete at Position
6. Delete at End
7. Traverse
8. Exit
Enter your choice: Enter value:
--- Singly Linked List Menu ---
1. Insert at Beginning
2. Insert at Position
3. Insert at End
4. Delete at Beginning
5. Delete at Position
6. Delete at End
7. Traverse
8. Exit
Enter your choice: Enter value and position (0-indexed):
--- Singly Linked List Menu ---
1. Insert at Beginning
2. Insert at Position
3. Insert at End
4. Delete at Beginning
5. Delete at Position
6. Delete at End
7. Traverse
8. Exit
Enter your choice: Linked List: 10 -> 15 -> 20 -> 30 -> NULL

--- Singly Linked List Menu ---
1. Insert at Beginning
2. Insert at Position
3. Insert at End
4. Delete at Beginning
5. Delete at Position
6. Delete at End
7. Traverse
8. Exit
Enter your choice: Enter position (0-indexed):
--- Singly Linked List Menu ---
1. Insert at Beginning
2. Insert at Position
3. Insert at End
4. Delete at Beginning
5. Delete at Position
6. Delete at End
7. Traverse
8. Exit
Enter your choice: Linked List: 10 -> 20 -> 30 -> NULL

--- Singly Linked List Menu ---
1. Insert at Beginning
2. Insert at Position
3. Insert at End
4. Delete at Beginning
5. Delete at Position
6. Delete at End
7. Traverse
8. Exit
Enter your choice: Exiting program.
```

Verification: Program compiled successfully and the output shown was captured from its execution using the test case printed above the output area.

EXPERIMENT 08 — Implementation of Circular Queue using Linked List

Source filename: 11259A477exp8.c

Aim: To write a C program to implement a circular queue using a singly linked list and to perform ENQUEUE, DEQUEUE, and display operations.

Corrected C source code

```
#include <stdio.h>
 #include <stdlib.h>

 struct Node {
     int data;
     struct Node *next;
 };

 struct Node *rear = NULL; /* rear->next points to front (circular) */

 void enqueue(int val) {
     struct Node *newNode = (struct Node *) malloc(sizeof(struct Node));
     newNode->data = val;
     if (rear == NULL) {
         newNode->next = newNode;

             rear = newNode;
         } else {
             newNode->next = rear->next;
             rear->next = newNode;
             rear = newNode;
         }
         printf("%d enqueued to circular queue.\n", val);
 }

 void dequeue() {
     if (rear == NULL) {
         printf("Queue Underflow\n");
         return;
     }
     struct Node *front = rear->next;
     printf("%d dequeued from circular queue.\n", front->data);
     if (front == rear) {
         rear = NULL;
     } else {
         rear->next = front->next;
     }
     free(front);
 }

 void display() {
     if (rear == NULL) {
          printf("Queue is empty.\n");
          return;
     }
     struct Node *temp = rear->next; /* front */
     printf("Circular Queue elements: ");
     do {
          printf("%d ", temp->data);
          temp = temp->next;
     } while (temp != rear->next);
     printf("\n");
 }

 int main() {
     int choice, val;
     do {
          printf("\n--- Circular Queue (Linked List) Menu ---\n");
          printf("1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");
          printf("Enter your choice: ");
          scanf("%d", &choice);

             switch (choice) {
                 case 1:
                     printf("Enter value to enqueue: ");
                     scanf("%d", &val);
                     enqueue(val);
                     break;
                 case 2:
                     dequeue();
                     break;
                 case 3:
                     display();
                     break;
                 case 4:
```

```
                printf("Exiting program.\n");
                break;
            default:
                printf("Invalid choice.\n");
        }
    } while (choice != 4);
    return 0;
}
```

EXPERIMENT 08 — RUN OUTPUT

Output screenshot name: 11259A477exp8_output.png

OUTPUT | 11259A477 exp8.c

Test case: Enqueue 10, 20, 30 | Display | Dequeue | Display

```
--- Circular Queue (Linked List) Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: Enter value to enqueue: 10 enqueued to circular queue.

--- Circular Queue (Linked List) Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: Enter value to enqueue: 20 enqueued to circular queue.

--- Circular Queue (Linked List) Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: Enter value to enqueue: 30 enqueued to circular queue.

--- Circular Queue (Linked List) Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: Circular Queue elements: 10 20 30

--- Circular Queue (Linked List) Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 10 dequeued from circular queue.

--- Circular Queue (Linked List) Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: Circular Queue elements: 20 30

--- Circular Queue (Linked List) Menu ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: Exiting program.
```

Verification: Program compiled successfully and the output shown was captured from its execution using the test case printed above the output area.

EXPERIMENT 09 — Implementation and Traversal (In-order, Pre-order, Post-order) of a Binary Search Tree

Source filename: 11259A477exp9.c

Aim: To write a C program to implement a Binary Search Tree (BST) and to perform insertion and traversal of the tree using in-order, pre-order, post-order, and level-order techniques.

Corrected C source code

```
#include <stdio.h>
 #include <stdlib.h>

 struct Node {
     int data;
     struct Node *left, *right;
 };

 struct Node *createNode(int val) {
     struct Node *newNode = (struct Node *) malloc(sizeof(struct Node));
     newNode->data = val;
     newNode->left = newNode->right = NULL;
     return newNode;
 }

 struct Node *insert(struct Node *root, int val) {
     if (root == NULL)
         return createNode(val);
     if (val < root->data)
         root->left = insert(root->left, val);
     else if (val > root->data)
         root->right = insert(root->right, val);
     return root;
 }

 void inorder(struct Node *root) {
     if (root != NULL) {
         inorder(root->left);
         printf("%d ", root->data);
         inorder(root->right);
     }
 }

 void preorder(struct Node *root) {
     if (root != NULL) {
         printf("%d ", root->data);
         preorder(root->left);
         preorder(root->right);
     }
 }

 void postorder(struct Node *root) {
     if (root != NULL) {
         postorder(root->left);
         postorder(root->right);
         printf("%d ", root->data);
     }
 }

 #define QMAX 100
 void levelorder(struct Node *root) {
     struct Node *queue[QMAX];
     int front = 0, rear = 0;

         if (root == NULL) {
             printf("Tree is empty.\n");
             return;
         }
         queue[rear++] = root;
         while (front < rear) {
             struct Node *curr = queue[front++];
             printf("%d ", curr->data);
             if (curr->left != NULL)
                 queue[rear++] = curr->left;
             if (curr->right != NULL)
                 queue[rear++] = curr->right;
         }
 }

 int main() {
     struct Node *root = NULL;
     int n, i, val, choice;

         printf("Enter number of nodes to insert: ");
         scanf("%d", &n);
```

```
        printf("Enter %d values: ", n);
        for (i = 0; i < n; i++) {
            scanf("%d", &val);
            root = insert(root, val);
        }

        do {
        printf("\n--- BST Traversal Menu ---\n");
        printf("1. In-order\n2. Pre-order\n3. Post-order\n4. Level-order\n5. Insert new node\n6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

               switch (choice) {
                   case 1:
                       printf("In-order Traversal: ");
                       inorder(root);
                       printf("\n");
                       break;
                   case 2:
                       printf("Pre-order Traversal: ");
                       preorder(root);
                       printf("\n");
                       break;

                case 3:
                    printf("Post-order Traversal: ");
                    postorder(root);
                    printf("\n");
                    break;
                case 4:
                    printf("Level-order Traversal: ");
                    levelorder(root);
                    printf("\n");
                    break;
                case 5:
                    printf("Enter value to insert: ");
                    scanf("%d", &val);
                    root = insert(root, val);
                    break;
                case 6:
                    printf("Exiting program.\n");
                    break;
                default:
                    printf("Invalid choice.\n");
            }
        } while (choice != 6);
        return 0;
}
```

EXPERIMENT 09 — RUN OUTPUT

Output screenshot name: 11259A477exp9_output.png

OUTPUT | 11259A499exp9.c

```
Test case: n = 7 | Values: 50 30 70 20 40 60 80 | Run all four traversals

Enter number of nodes to insert: Enter 7 values:
--- BST Traversal Menu ---
1. In-order
2. Pre-order
3. Post-order
4. Level-order
5. Insert new node
6. Exit
Enter your choice: In-order Traversal: 20 30 40 50 60 70 80

--- BST Traversal Menu ---
1. In-order
2. Pre-order
3. Post-order
4. Level-order
5. Insert new node
6. Exit
Enter your choice: Pre-order Traversal: 50 30 20 40 70 60 80

--- BST Traversal Menu ---
1. In-order
2. Pre-order
3. Post-order
4. Level-order
5. Insert new node
6. Exit
Enter your choice: Post-order Traversal: 20 40 30 60 80 70 50

--- BST Traversal Menu ---
1. In-order
2. Pre-order
3. Post-order
4. Level-order
5. Insert new node
6. Exit
Enter your choice: Level-order Traversal: 50 30 70 20 40 60 80

--- BST Traversal Menu ---
1. In-order
2. Pre-order
3. Post-order
4. Level-order
5. Insert new node
6. Exit
Enter your choice: Exiting program.
```

Verification: Program compiled successfully and the output shown was captured from its execution using the test case printed above the output area.

EXPERIMENT 10 — Implementation of Graph Search Algorithms (BFS and DFS)

Source filename: 11259A477exp10.c

Aim: To write a C program to represent a graph using an adjacency matrix and to traverse it using the Breadth-First Search (BFS) and Depth-First Search (DFS) algorithms.

Corrected C source code

```
#include <stdio.h>

#define MAX 20

int adj[MAX][MAX];
int visited[MAX];
int n;

void bfs(int start) {
    int queue[MAX], front = 0, rear = 0, i;
    for (i = 0; i < n; i++)
        visited[i] = 0;

        visited[start] = 1;
        queue[rear++] = start;

        printf("BFS Traversal: ");

        while (front < rear) {
            int curr = queue[front++];
            printf("%d ", curr);
            for (i = 0; i < n; i++) {
                if (adj[curr][i] == 1 && !visited[i]) {
                    visited[i] = 1;
                    queue[rear++] = i;
                }
            }
        }
        printf("\n");
}

void dfsUtil(int v) {
    int i;
    visited[v] = 1;
    printf("%d ", v);
    for (i = 0; i < n; i++) {
        if (adj[v][i] == 1 && !visited[i])
            dfsUtil(i);
    }
}

void dfs(int start) {
    int i;
    for (i = 0; i < n; i++)
        visited[i] = 0;
    printf("DFS Traversal: ");
    dfsUtil(start);
    printf("\n");
}

int main() {
    int edges, i, u, v, start, choice;

        printf("Enter number of vertices: ");
        scanf("%d", &n);
        printf("Enter number of edges: ");
        scanf("%d", &edges);

        for (i = 0; i < n; i++)
            for (v = 0; v < n; v++)
                adj[i][v] = 0;

        printf("Enter edges (u v) for an undirected graph:\n");

        for (i = 0; i < edges; i++) {
            scanf("%d %d", &u, &v);
            adj[u][v] = 1;
            adj[v][u] = 1;
        }

        printf("Enter starting vertex: ");
        scanf("%d", &start);

        do {
               printf("\n--- Graph Traversal Menu ---\n");
```

```
            printf("1. BFS\n2. DFS\n3. Exit\n");
            printf("Enter your choice: ");
            scanf("%d", &choice);

        switch (choice) {
            case 1:
                bfs(start);
                break;
            case 2:
                dfs(start);
                break;
            case 3:
                printf("Exiting program.\n");
                break;
            default:
                printf("Invalid choice.\n");
        }
    } while (choice != 3);
    return 0;
}
```

EXPERIMENT 10 — RUN OUTPUT

Output screenshot name: 11259A477exp10_output.png

OUTPUT | 11259A477 exp10.c

Test case: Vertices = 5 | Edges: (0,1), (0,2), (1,3), (1,4), (2,4) | Start = 0 | BFS and DFS

```
Enter number of vertices: Enter number of edges: Enter edges (u v) for an undirected graph:
Enter starting vertex:
--- Graph Traversal Menu ---
1. BFS
2. DFS
3. Exit
Enter your choice: BFS Traversal: 0 1 2 3 4

--- Graph Traversal Menu ---
1. BFS
2. DFS
3. Exit
Enter your choice: DFS Traversal: 0 1 3 4 2

--- Graph Traversal Menu ---
1. BFS
2. DFS
3. Exit
Enter your choice: Exiting program.
```

Verification: Program compiled successfully and the output shown was captured from its execution using the test case printed above the output area.

EXPERIMENT 11 — Implementation of Bubble, Selection and Heap Sorts

Source filename: 11259A477exp11.c

Aim: To write a C program to sort a given list of elements using Bubble Sort, Selection Sort, and Heap Sort techniques.

Corrected C source code

```
#include <stdio.h>

void printArray(int arr[], int n) {
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

void bubbleSort(int arr[], int n) {
    int i, j, temp;

        for (i = 0; i < n - 1; i++) {
            for (j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
}

void selectionSort(int arr[], int n) {
    int i, j, minIndex, temp;
    for (i = 0; i < n - 1; i++) {
        minIndex = i;
        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex])
                minIndex = j;
        }
        temp = arr[minIndex];
        arr[minIndex] = arr[i];
        arr[i] = temp;
    }
}

void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    int temp;

        if (left < n && arr[left] > arr[largest])
            largest = left;
        if (right < n && arr[right] > arr[largest])
            largest = right;

        if (largest != i) {
            temp = arr[i];
            arr[i] = arr[largest];
            arr[largest] = temp;
            heapify(arr, n, largest);
        }
}

void heapSort(int arr[], int n) {

        int i, temp;
        for (i = n / 2 - 1; i >= 0; i--)
            heapify(arr, n, i);

        for (i = n - 1; i > 0; i--) {
            temp = arr[0];
            arr[0] = arr[i];
            arr[i] = temp;
            heapify(arr, i, 0);
        }
}

int main() {
    int arr[100], temp[100], n, i, choice;

        printf("Enter number of elements: ");
        scanf("%d", &n);
        printf("Enter %d elements: ", n);
```

```
        for (i = 0; i < n; i++)
            scanf("%d", &arr[i]);

        do {
        printf("\n--- Sorting Menu ---\n");
        printf("1. Bubble Sort\n2. Selection Sort\n3. Heap Sort\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

               for (i = 0; i < n; i++)
                   temp[i] = arr[i];

               switch (choice) {
                   case 1:
                       bubbleSort(temp, n);
                       printf("Sorted array (Bubble Sort): ");
                       printArray(temp, n);
                       break;
                   case 2:
                       selectionSort(temp, n);
                       printf("Sorted array (Selection Sort): ");
                       printArray(temp, n);
                       break;
                   case 3:
                       heapSort(temp, n);
                       printf("Sorted array (Heap Sort): ");

                    printArray(temp, n);
                    break;
                case 4:
                    printf("Exiting program.\n");
                    break;
                default:
                    printf("Invalid choice.\n");
            }
        } while (choice != 4);
        return 0;
}
```

EXPERIMENT 11 — RUN OUTPUT

Output screenshot name: 11259A477exp11_output.png

OUTPUT | 11259A477 exp11.c

```
Test case: n = 6 | Values: 45 12 78 3 89 21 | Bubble, Selection and Heap sort

Enter number of elements: Enter 6 elements:
--- Sorting Menu ---
1. Bubble Sort
2. Selection Sort
3. Heap Sort
4. Exit
Enter your choice: Sorted array (Bubble Sort): 3 12 21 45 78 89

--- Sorting Menu ---
1. Bubble Sort
2. Selection Sort
3. Heap Sort
4. Exit
Enter your choice: Sorted array (Selection Sort): 3 12 21 45 78 89

--- Sorting Menu ---
1. Bubble Sort
2. Selection Sort
3. Heap Sort
4. Exit
Enter your choice: Sorted array (Heap Sort): 3 12 21 45 78 89

--- Sorting Menu ---
1. Bubble Sort
2. Selection Sort
3. Heap Sort
4. Exit
Enter your choice: Exiting program.
```

Verification: Program compiled successfully and the output shown was captured from its execution using the test case printed above the output area.

EXPERIMENT 12 — Implementation of Hashing with Collision Resolution using Linear Probing

Source filename: 11259A477exp12.c

Aim: To write a C program to implement a hash table using the division hash function and to resolve collisions using the linear probing technique, and to perform insertion, search, and display operations.

Corrected C source code

```
#include <stdio.h>

 #define SIZE 10

 int hashTable[SIZE];
 int isOccupied[SIZE] = {0};

 int hashFunction(int key) {
     return key % SIZE;
 }

 void insert(int key) {
     int index = hashFunction(key);
     int start = index;
     int count = 0;

         while (isOccupied[index] == 1) {
             if (hashTable[index] == key) {
                 printf("Key %d already exists.\n", key);
                 return;
             }
             index = (index + 1) % SIZE;
             count++;
             if (count == SIZE) {
                 printf("Hash Table is full. Cannot insert %d\n", key);
                 return;
             }
         }
         hashTable[index] = key;
         isOccupied[index] = 1;
         if (index != start)
             printf("Collision occurred for key %d. ", key);
         printf("Key %d inserted at index %d\n", key, index);
 }

 void search(int key) {
     int index = hashFunction(key);
     int start = index;
     int count = 0;

         while (isOccupied[index] == 1) {
             if (hashTable[index] == key) {
                 printf("Key %d found at index %d\n", key, index);
                 return;
             }
             index = (index + 1) % SIZE;
             count++;
             if (count == SIZE)
                 break;
         }
         printf("Key %d not found in hash table.\n", key);
 }

 void display() {
     int i;
     printf("\nHash Table:\n");
     for (i = 0; i < SIZE; i++) {
         if (isOccupied[i])
              printf("Index %d: %d\n", i, hashTable[i]);
         else
              printf("Index %d: --\n", i);

         }
 }

 int main() {
     int choice, key;
     do {
          printf("\n--- Hashing (Linear Probing) Menu ---\n");
          printf("1. Insert\n2. Search\n3. Display\n4. Exit\n");
          printf("Enter your choice: ");
          scanf("%d", &choice);

             switch (choice) {
                 case 1:
```

```
                printf("Enter key to insert: ");
                scanf("%d", &key);
                insert(key);
                break;
            case 2:
                printf("Enter key to search: ");
                scanf("%d", &key);
                search(key);
                break;
            case 3:
                display();
                break;
            case 4:
                printf("Exiting program.\n");
                break;
            default:
                printf("Invalid choice.\n");
        }
    } while (choice != 4);
    return 0;
}
```

EXPERIMENT 12 — RUN OUTPUT

Output screenshot name: 11259A477exp12_output.png

OUTPUT | 11259A477 exp12.c

Test case: Insert keys 25, 35, 15 | Display table | Search 35

```
--- Hashing (Linear Probing) Menu ---
1. Insert
2. Search
3. Display
4. Exit
Enter your choice: Enter key to insert: Key 25 inserted at index 5

--- Hashing (Linear Probing) Menu ---
1. Insert
2. Search
3. Display
4. Exit
Enter your choice: Enter key to insert: Collision occurred for key 35. Key 35 inserted at index 6

--- Hashing (Linear Probing) Menu ---
1. Insert
2. Search
3. Display
4. Exit
Enter your choice: Enter key to insert: Collision occurred for key 15. Key 15 inserted at index 7

--- Hashing (Linear Probing) Menu ---
1. Insert
2. Search
3. Display
4. Exit
Enter your choice:
Hash Table:
Index 0: --
Index 1: --
Index 2: --
Index 3: --
Index 4: --
Index 5: 25
Index 6: 35
Index 7: 15
Index 8: --
Index 9: --

--- Hashing (Linear Probing) Menu ---
1. Insert
2. Search
3. Display
4. Exit
Enter your choice: Enter key to search: Key 35 found at index 6

--- Hashing (Linear Probing) Menu ---
1. Insert
2. Search
3. Display
4. Exit
Enter your choice: Exiting program.
```

Verification: Program compiled successfully and the output shown was captured from its execution using the test case printed above the output area.